# Lecture 5: Graphs

- Often distance in a network are much more relevant

## Graphs

- Graphs are mathematical models for networks
  - streets, rivers, railway networks
  - boundaries
  - areas and their adjacency relationships
  - other examples
    - internet, social networks

## Definition

- Graph consists of:
  - Set $V$ of nodes (or vertices)
    - $V = \{1, 2, 4, 6, 9\}$
  - Set $E$ of edges - each edge $e \in E$ is set of 2 nodes
    - $E \subseteq \{\{u, v\} | u, v \in V, u \neq v\}$
    - $E = \{\{1, 2\}, \{2, 4\}, \{1, 6\}, \{1, 4\}, \{2, 9\}\}$
- Representation with different drawings

### Adjacency, Incidence, Degree

- Adjacency : **neighbour relationship** of 2 nodes u,v via an edge
- Incidence : **relationship** between a node $u$ and an edge $e = \{u, v\}$
- Degree of *node*: number of its incident edges. ($degree(u) = 4$)
- Degree of *graph*: Max node degree:
  - $degree(G) = max\{degree(u) | u \in V\}$

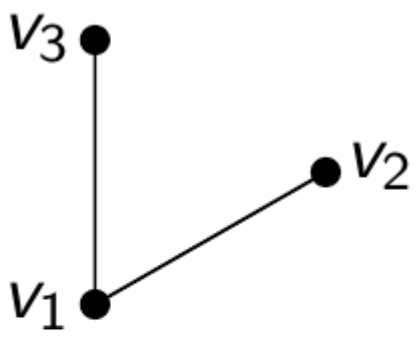
## Directed Graph / Digraph

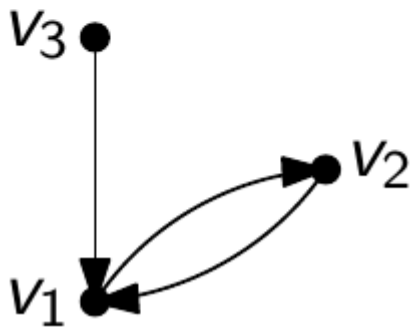
- Directed Graph consists of:
  - Set $V$ of nodes (or vertices)
    - $V = \{1, 2, 4, 6, 9\}$
  - Set $A$ of directed edges/arcs - each edge $3 \in E$ is set of 2 nodes
    - $E \subseteq \{(u, v) | u, v \in V, u \neq v\}$

    - $E = \{(1,2),(2,4),(1,6),(1,4),(2,9)\}$
- Representation with different drawings with vertices as directed arrows
- **Attention:** For pairs of $(a,b)$ and $(b,a)$ with $a \neq b$ it holds that $(a,b)\neq(b,a).
  - For 2 sets $\{a,b\}$ it holds though

## Representations of Graphs

- Adjacency Lists
  - Each node is an object, storing reference to all of its neighbors
    - `Vertex` class with *at least* follwowing attributes
      - id,
      - and list of neighbours
- Incidence Lists
  - Each node/each edge is an object
  - Nodes store reference to list with incident edges.
    - additional class Edge with attributes:
      - Origin Vertex
      - Destination Vertex
      - length
- Adjacency Matrix
  - Nodes are stored in array `vertices[1..n]`
  - Adjacency matrix $a$ of dimension $n \times n$ defines edges
  - `a[i][j] = 1` $\Leftrightarrow$ `vertices[i] is neighbor of vertices[j]`
  - a is symmetric for undirected graphs:
    - $a = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$
  - For directed graphs, $a$ can be asymmetric:
    - $a = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$




- Often implemented with boolean instead of 1/0

**Running times of important queries**

| Operation | adjacency lists | adjacency matrix |
|---|---|---|
| Find all neighbours of $u$ | $\mathcal{O}(degree(u))$ | $\mathcal{O}(len(V))$ |
| Is $v$ neighbour of $U$? | $\mathcal{O}(degree(u))$ | $\mathcal{O}(1)$ |
| Space requirement | $\mathcal{O}(len(V) + len(E))$ | $\mathcal{O}(len(V)^2)$ |

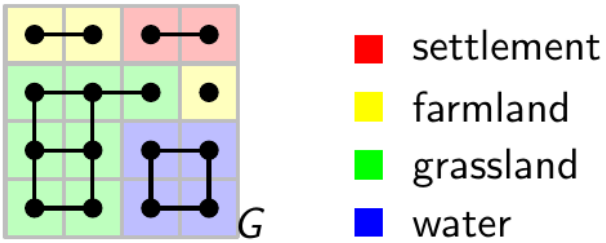
- Adjacency matrix good for **dense graphs**!

# Algorithms for Graphs

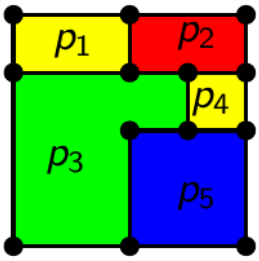
## Connected Components

- Find partition of $G$ into minimum number of connected subgraphs

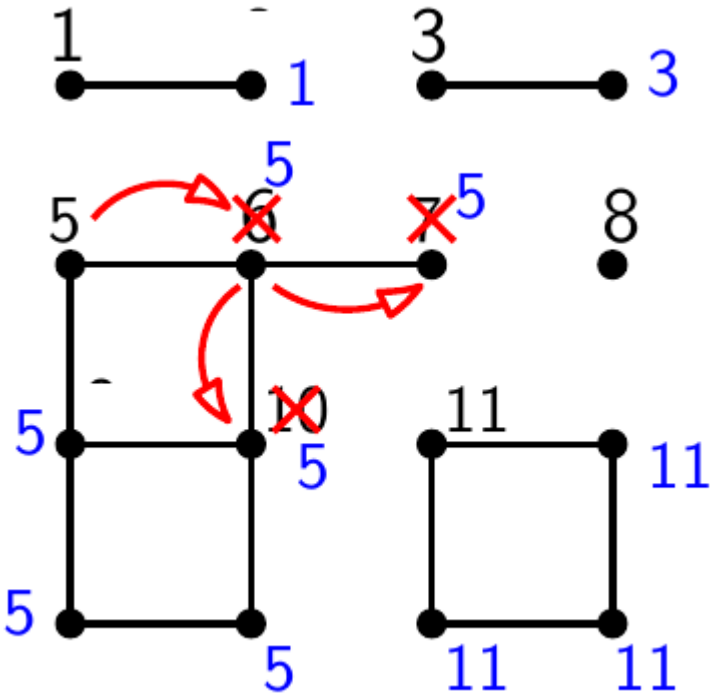
We are given a land-cover map in a raster format (pixels).




Compute a vector representation of the map (polygons).

- 


  Each polygon corresponds to a connected component of the graph $G$ that contains a node for each pixel and an edge for each two adjacent pixels that have the same color.

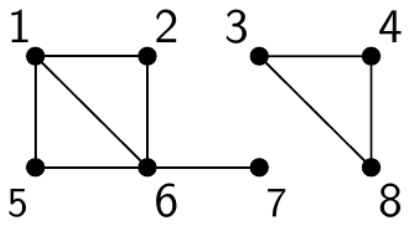
- Each node has an attribute **centre** (initialised with the node's *id*)
- Start at **seed node** $s$, the *centre*-value of all unvisited adjacent nodes is overwritten with *center*-value of $s$
- -> **Recursive** propagation thorugh all adjacent nodes
- -> All nodes within *same* connected component: Same *centre*-value
- For Polygons: follow boundaries of connected components
- 


**Depth First Search**

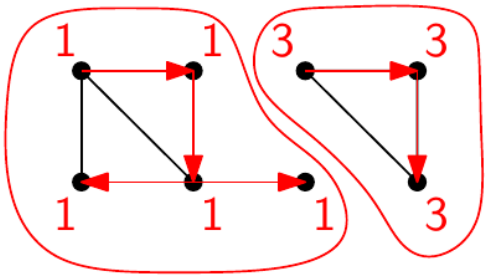
```
DepthFirstSearch(Graph G = (V, E))
  foreach u ∈ V do
    u.visited = false
  foreach u ∈ V do
    DepthFirstSearchVisit(u, u.id)

DepthFirstSearchVisit(Node u, int id)
  if u.visited == false then
    u.visited = true
    u.center = id
    foreach v ∈ u.neighbors do
      DepthFirstSearchVisit(v, id)
```

graph with node IDs:




followed arcs & center IDs:




connected components

---

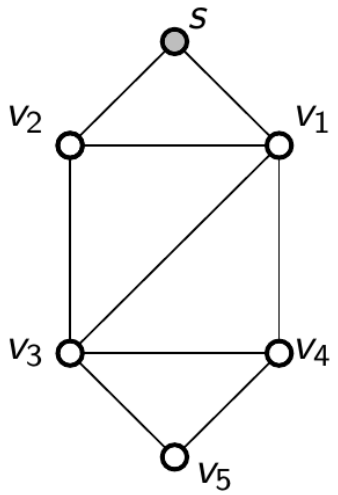
## Shortest Paths in Unweighted Graphs

- Given
  - Graph $G = (V, E)$
  - edge weighting w: $E \to \mathbb{R}_{\geq 0}$
  - Source $s \in V$ and destination $t \in V$
- Solution:
  - weight minimal s-t-path in G
- Unweighted Graphs: $w(e) = 1$ for all $e \in E$
- Algorithm: Breadth First Search
  -

```
Initialize(Graph G, Vertex s)
  foreach u ∈ V do
    u.color = white     not visited yet
    u.d = ∞             min. known distance from s
    u.π = nil           predecessor on shortest
  s.color = gray        known path from s
  s.d = 0               only s lies on frontier
```

# Breadth First Search

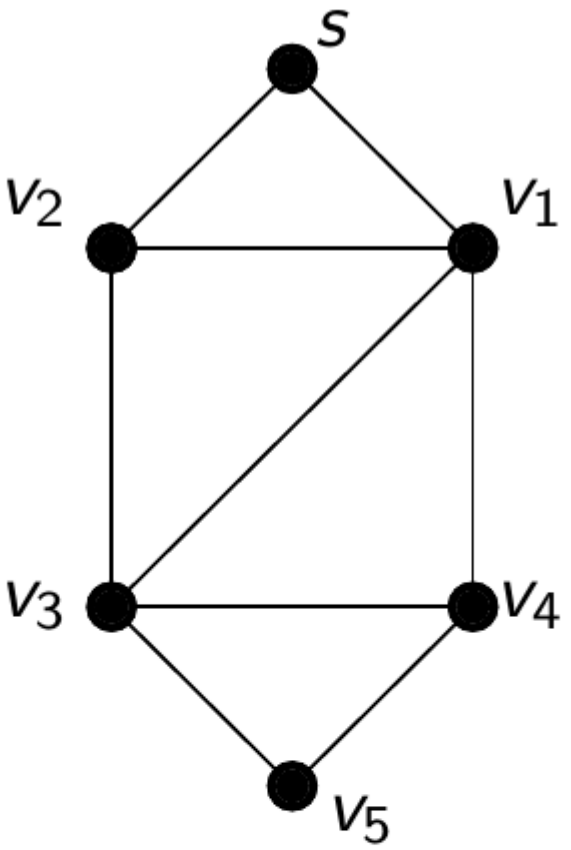
```
BFS(Graph G = (V, E), Vertex s)
  Initialize(G, s)
  Q = new Queue()
  Q.Enqueue(s)
  while Q ≠ ∅ do
    u = Q.Dequeue()
    foreach v ∈ u.neighbors do
      if v.color == white then
        v.color = gray
        v.d = u.d + 1
        v.π = u
        Q.Enqueue(v)
    u.color = black
```




$Q = \emptyset$

$u = s$

| | $s$ | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ |
|---|---|---|---|---|---|---|
| $d$ | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\pi$ | $nil$ | $nil$ | $nil$ | $nil$ | $nil$ | $nil$ |




$$Q = \emptyset$$

$$u = v_5$$

| | $s$ | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ |
|---|---|---|---|---|---|---|
| $d$ | 0 | 1 | 1 | 2 | 2 | 3 |
| $\pi$ | $nil$ | $s$ | $s$ | $v_1$ | $v_1$ | $v_3$ |

## Summary

- Graph is a mathematical concept of a network
- Depth First Search (DFS) and Breadth First search (BFS) are algorithms for traversing a graph $G = (v, E)$, both with runtime $\mathcal{O}(|V| + |E|)$
- Both can be used to find connected components
- BFS used to fid shortest paths in **unweighted graphs**